Laboratory Exercise 6

By: Anderson

Part I

Schematic

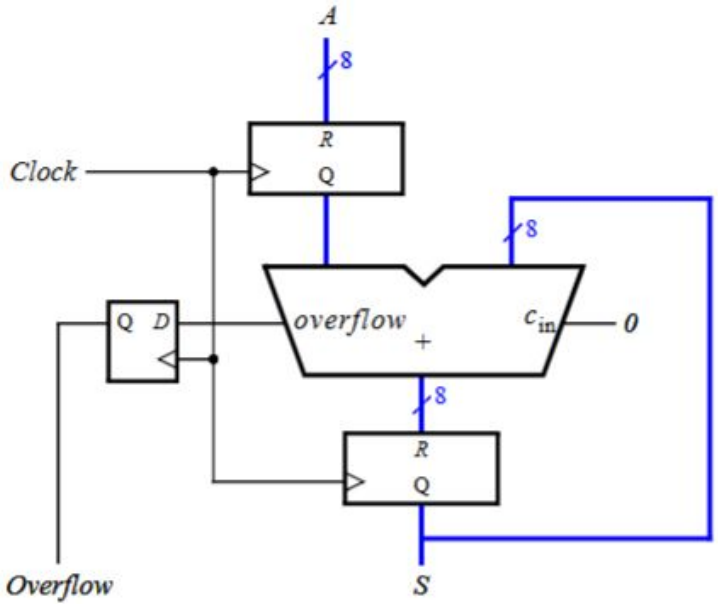
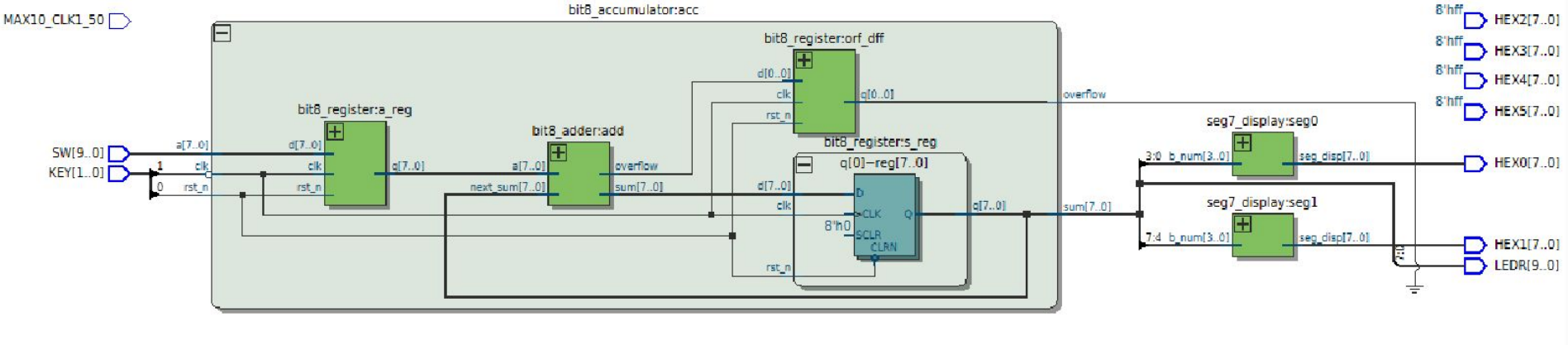
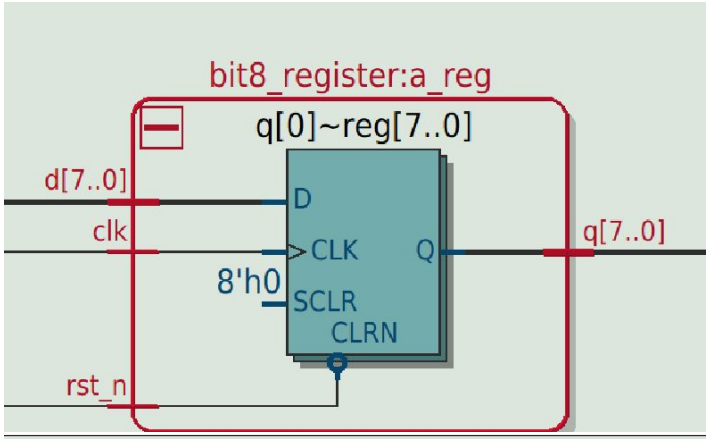


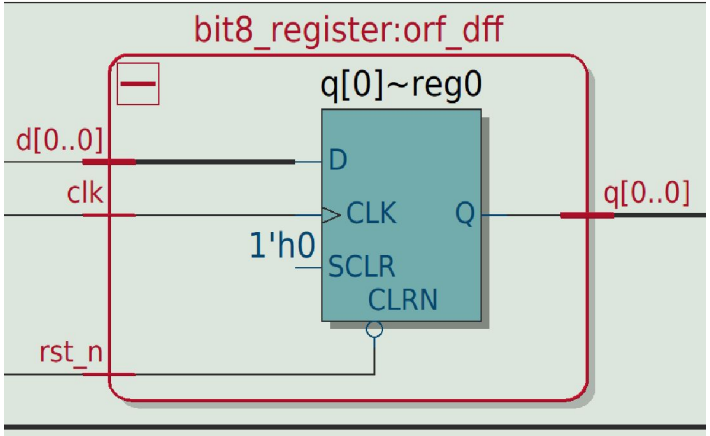
Figure 2: An eight-bit accumulator circuit.

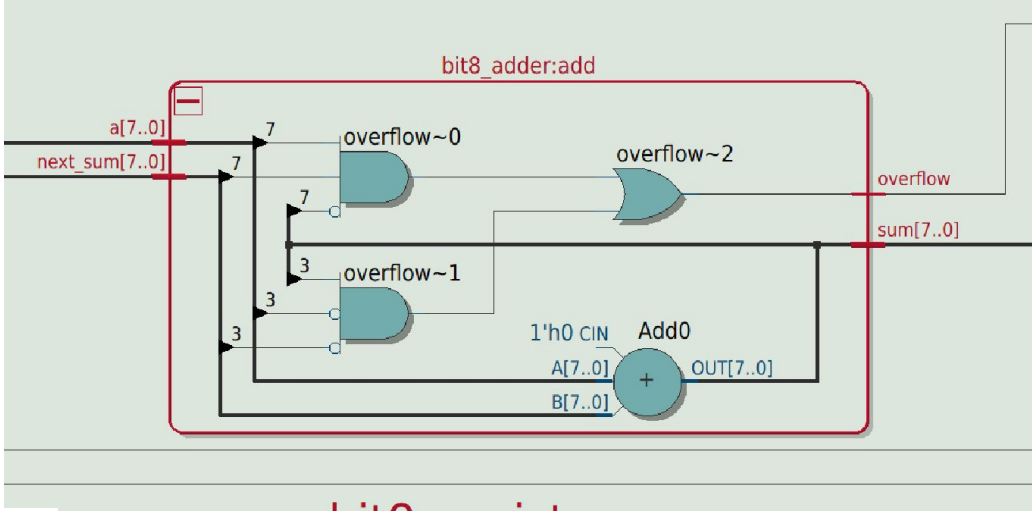
RTL Viewer

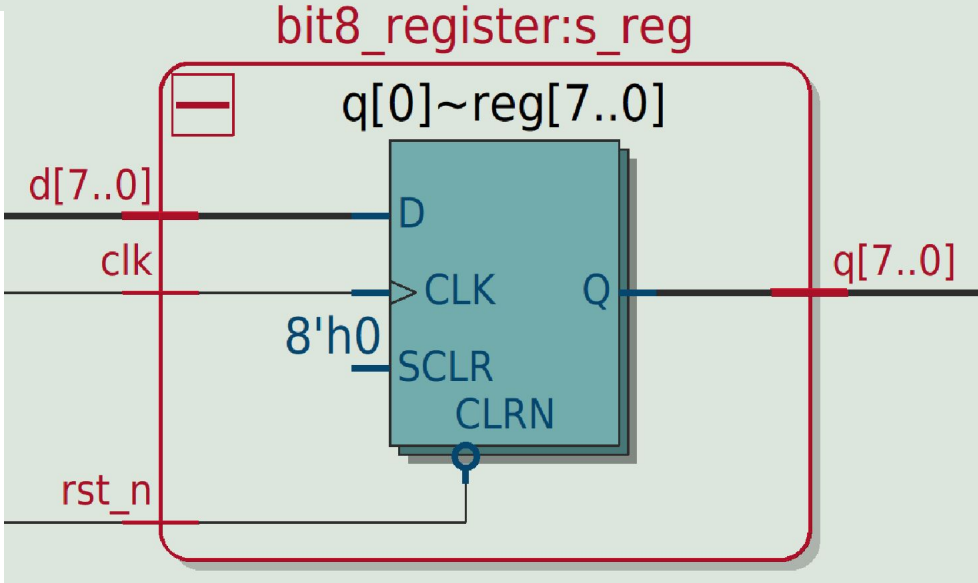

MAX10_CLK1_50
SW[9..0]
KEY[1..0]
bit8_accumulator:acc
bit8_register:a_reg
bit8_adder:add
bit8_register:orf_dff
bit8_register:s_reg
q[0]~reg[7..0]
seg7_display:seg0
seg7_display:seg1
overflow
sum[7..0]
HEX0[7..0]
HEX1[7..0]
HEX2[7..0]
HEX3[7..0]
HEX4[7..0]
HEX5[7..0]
LEDR[9..0]

RTL Viewer


bit8_register:a_reg
q[0]~reg[7..0]
d[7..0]
clk
8'h0
D
CLK
Q
SCLR
CLRN
q[7..0]
rst_n


bit8_register:orf_dff
q[0]~reg0
d[0..0]
clk
1'h0
D
CLK
Q
SCLR
CLRN
q[0..0]
rst_n


bit8_adder:add
a[7..0]
next_sum[7..0]
7
overflow~0
overflow~2
overflow
sum[7..0]
3
overflow~1
1'h0 CIN
Add0
A[7..0]
B[7..0]
OUT[7..0]


bit8_register:s_reg
q[0]~reg[7..0]
d[7..0]
clk
8'h0
D
CLK
Q
SCLR
CLRN
q[7..0]
rst_n

Verilog Code

```verilog
module bit8_adder (
    input [7:0] a, next_sum,
    output [7:0] sum,
    output overflow
);
    wire c_o;
    assign {c_o, sum} = a + next_sum;

    assign overflow = (a[7] & next_sum[7] & ~sum[7]) | (~a[7] & ~next_sum[7] & sum[7]);

endmodule
```

```verilog
module bit8_register #(
    parameter bit = 8
)(
    input clk, rst_n,
    input [bit - 1:0] d,
    output reg [bit - 1:0] q
);

    always @(posedge clk or negedge rst_n) begin
        if (~rst_n) begin
            q <= {bit{1'b0}};
        end else begin
            q <= d;
        end
    end

endmodule
```

```verilog
module seg7_display(
    input [3:0] b_num,
    output reg [7:0] seg_disp
);

    always @(*) begin
        case(b_num)
            4'd0:  seg_disp = 8'b0100_0000; // Segments 0,1,2,3,4,5 ON  (g=1)
            4'd1:  seg_disp = 8'b0111_1001; // Segments 1,2 ON
            4'd2:  seg_disp = 8'b0010_0100; // Segments 0,1,6,4,3 ON
            4'd3:  seg_disp = 8'b0011_0000; // Segments 0,1,6,2,3 ON
            4'd4:  seg_disp = 8'b0001_1001; // Segments 5,6,1,2 ON
            4'd5:  seg_disp = 8'b0001_0010; // Segments 0,5,6,2,3 ON
            4'd6:  seg_disp = 8'b0000_0010; // Segments 0,5,4,3,2,6 ON
            4'd7:  seg_disp = 8'b0111_1000; // Segments 0,1,2 ON
            4'd8:  seg_disp = 8'b0000_0000; // Segments 0,1,2,3,4,5,6 ON
            4'd9:  seg_disp = 8'b0001_0000; // Segments 0,1,2,3,5,6 ON
            4'd10: seg_disp = 8'b0000_1000; // "A": Segments 0,1,2,4,5,6 ON
            4'd11: seg_disp = 8'b0000_0011; // "b": Segments 5,4,6,2,3 ON
            4'd12: seg_disp = 8'b0100_0110; // "C": Segments 0,5,4,3 ON
            4'd13: seg_disp = 8'b0010_0001; // "d": Segments 1,2,3,4,6 ON
            4'd14: seg_disp = 8'b0000_0110; // "E": Segments 0,5,6,4,3 ON
            4'd15: seg_disp = 8'b0000_1110; // "F": Segments 0,5,6,4 ON
            default: seg_disp = 8'b0111_1111; // All segments OFF
        endcase
    end

endmodule
```

Verilog Code

Works as intended however Input only outputs after clk has been pressed twice then once after that. This is most likely due to the first register causing a 2-cycle latency at the start. Another thing to notes is that the clk button has a debounce issue Causing the first two switches to cycle multiple times before outputting.

```
module bit8_accumulator (
    input clk, rst_n,
    input [7:0] a,
    output [7:0] sum,
    output overflow
);

    wire [7:0] r1_o, r2_o, add_o;
    wire ovf_o;

    bit8_register #(.bit(8)) a_reg (.clk(clk), .rst_n(rst_n), .d(a), .q(r1_o));
    bit8_adder add (.a(r1_o), .next_sum(r2_o), .sum(add_o), .overflow(ovf_o));
    bit8_register #(.bit(8)) s_reg (.clk(clk), .rst_n(rst_n), .d(add_o), .q(r2_o));

    bit8_register #(.bit(1)) orf_dff (.clk(clk), .rst_n(rst_n), .d(ovf_o), .q(overflow));

    assign sum = r2_o;

endmodule
```

```
module main(
    input    MAX10_CLK1_50,
    input    [9:0]    SW,
    input    [1:0]    KEY,
    output   [9:0]    LEDR,
    output   [7:0]    HEX0, HEX1, HEX2, HEX3, HEX4, HEX5

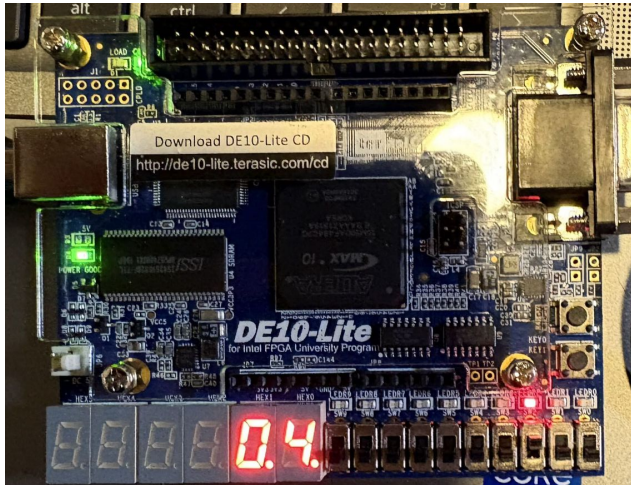
);
    //Part I
    bit8_accumulator acc(.clk(~KEY[1]), .rst_n(KEY[0]), .a(SW[7:0]), .sum(LEDR[7:0]), .overflow(LEDR[8]));

    seg7_display seg1 (.b_num(LEDR[7:4]), .seg_disp(HEX1));
    seg7_display seg0 (.b_num(LEDR[3:0]), .seg_disp(HEX0));

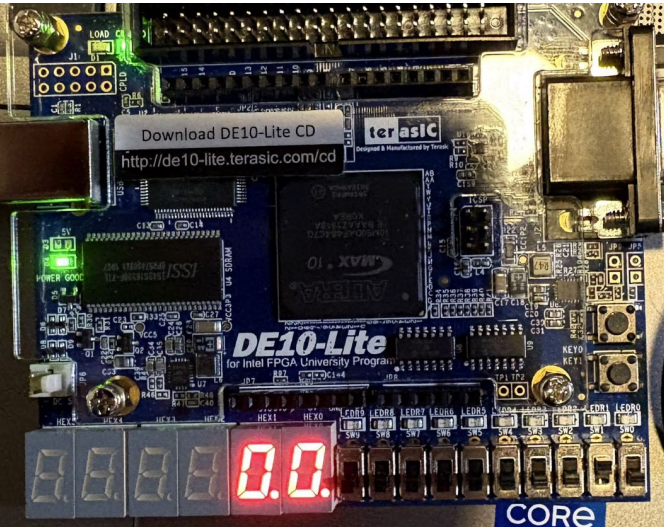
    assign HEX5 = 8'hFF;
    assign HEX4 = 8'hFF;
    assign HEX3 = 8'hFF;
    assign HEX2 = 8'hFF;

endmodule
```

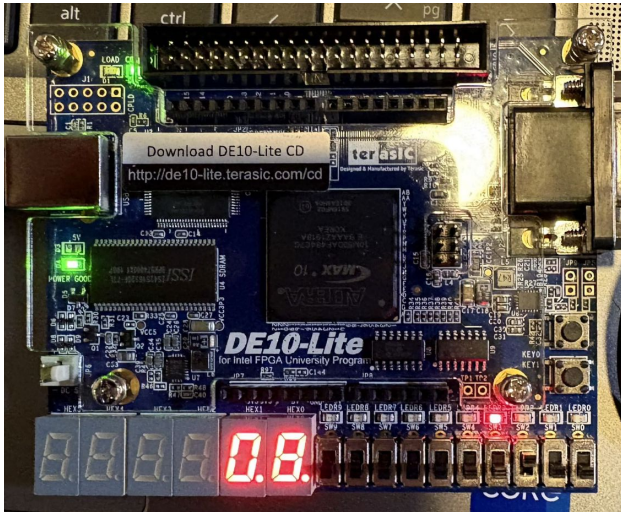
DE10-lite



SW2 on & KEY1 pressed



KEY1 pressed again



Part 2

Verilog Code

```
module bit8_accumulator (
    input clk, rst_n, add_sub,
    input [7:0] a,
    output [7:0] val,
    output overflow
);

    //Part I & II
    wire [7:0] r1_o, r2_o, arith_o;
    wire ovf_o;

    bit8_register #(.bit(8)) a_reg (.clk(clk), .rst_n(rst_n), .d(a), .q(r1_o));
// //Part I
// bit8_adder add (.a(r1_o), .next_sum(r2_o), .sum(arith_o), .overflow(ovf_o));
    //Part II
    bit8_arithmatic arith (.add_sub(add_sub), .a(r1_o), .next_val(r2_o), .val(arith_o), .overflow(ovf_o));
    bit8_register #(.bit(8)) s_reg (.clk(clk), .rst_n(rst_n), .d(arith_o), .q(r2_o));

    bit8_register #(.bit(1)) orf_dff (.clk(clk), .rst_n(rst_n), .d(ovf_o), .q(overflow));

    assign val = r2_o;

endmodule
```

```
);
    //Part I & II
    bit8_accumulator acc(.clk(~KEY[1]), .rst_n(KEY[0]), .add_sub(SW[9]), .a(SW[7:0]), .val(LEDR[7:0]), .overflow(LEDR[8]));

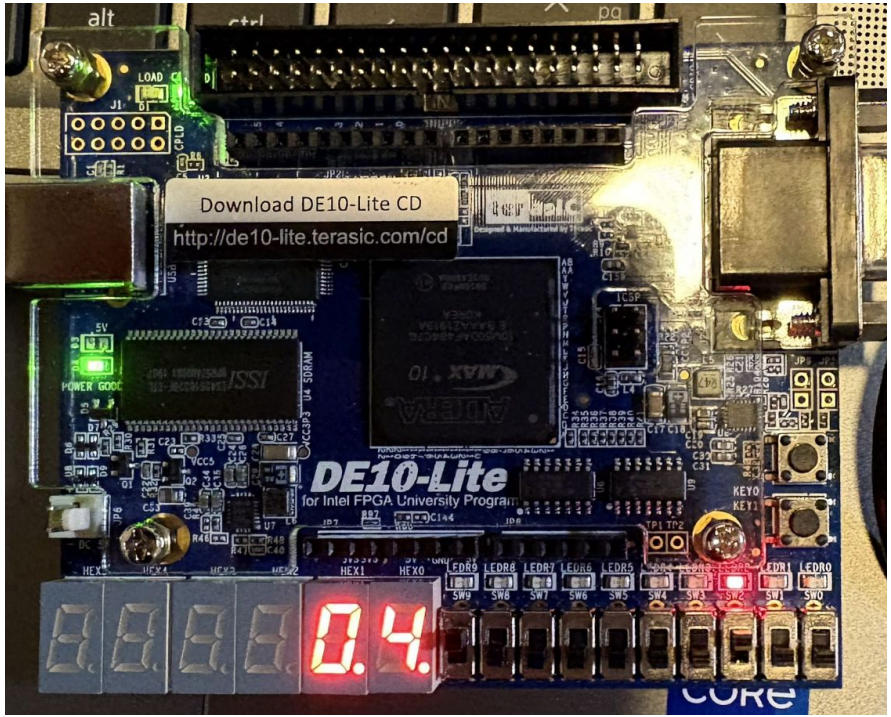
    seg7_display seg1 (.b_num(LEDR[7:4]), .seg_disp(HEX1));
    seg7_display seg0 (.b_num(LEDR[3:0]), .seg_disp(HEX0));

    assign HEX5 = 8'hFF;
    assign HEX4 = 8'hFF;
    assign HEX3 = 8'hFF;
    assign HEX2 = 8'hFF;
```

```
module bit8_arithmatic (
    input add_sub,
    input [7:0] a, next_val,
    output reg [7:0] val,
    output reg overflow
);

    always @(*) begin
        if (add_sub) begin
            val = next_val - a;
            overflow = (a[7] & ~next_val[7] & val[7]) | (~a[7] & next_val[7] & ~val[7]);
        end else begin
            val = next_val + a;
            overflow = (a[7] & next_val[7] & ~val[7]) | (~a[7] & ~next_val[7] & val[7]);
        end
    end
endmodule
```

DE10-lite



After SW9 on & KEY1 pressed from pt.1 state

Part 3

Schematic

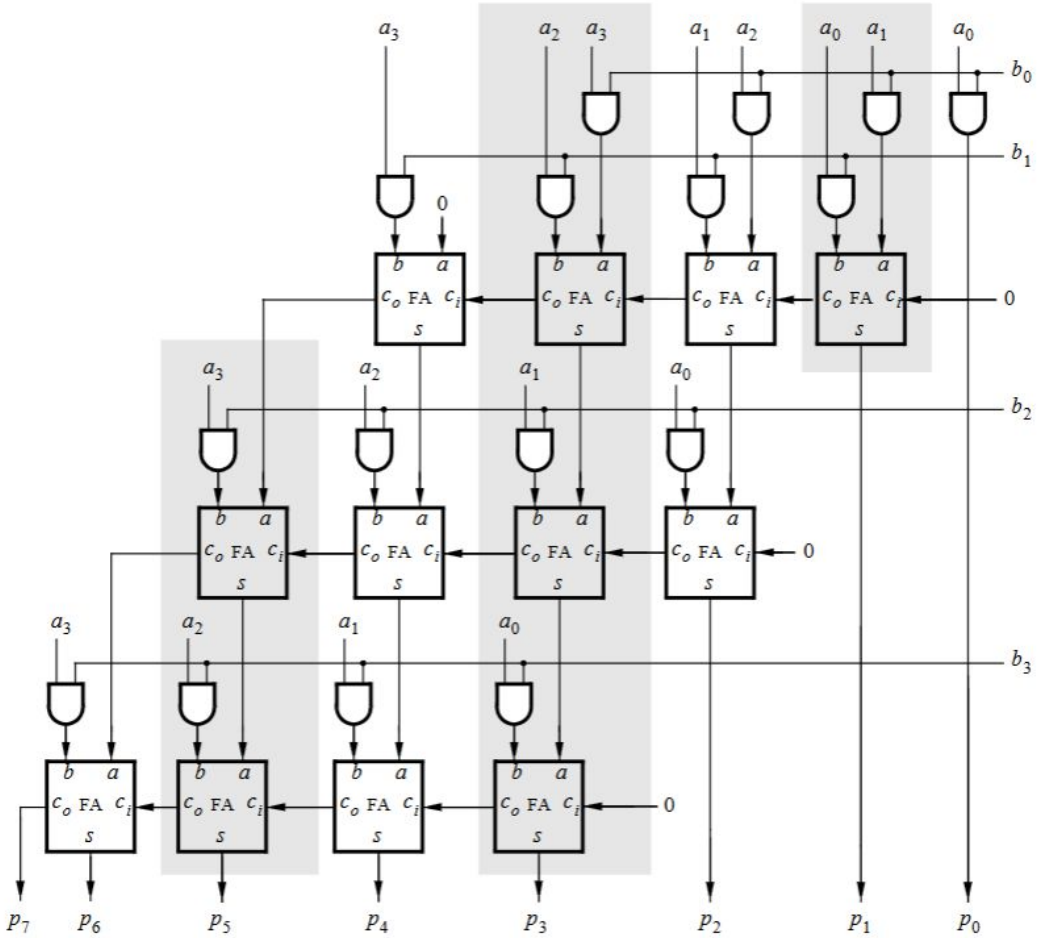


Figure 4: An array multiplier circuit.

Verilog Code

```verilog
module ripple_adder #(
    parameter WIDTH = 4
)(
    input [WIDTH-1:0] A, B,
    input C_i,
    output [WIDTH-1:0] Sum,
    output             C_o
);
    wire [WIDTH:0] c;

    assign c[0] = C_i;
    assign C_o = c[WIDTH];

    genvar i;
    generate
        for (i = 0; i < WIDTH; i = i + 1) begin: gen_fa
            assign Sum[i]  = A[i] ^ B[i] ^ c[i];
            assign c[i+1]  = (A[i] & B[i]) | (c[i] & (A[i] ^ B[i]));
        end
    endgenerate
endmodule
```

```verilog
module array_multi #(
    parameter n = 4
)(
    input [n-1:0] a, b,
    output [2*n-1:0] prod
);

    wire [n-1:0] s_row [0:n-1];
    wire [n-1:0] c_o;

    assign s_row[0] = a & {n{b[0]}};
    assign c_o[0] = 1'b0;
    assign prod[0] = s_row[0][0];

    genvar i;
    generate
        for (i = 1; i < n; i = i + 1) begin: gen_p
            ripple_adder #(.WIDTH(n)) row
            (.A({c_o[i-1], s_row[i-1][n-1:1]}), .B(a & {n{b[i]}}), .Sum(s_row[i]), .C_o(c_o[i]));

            assign prod[i] = s_row[i][0];
        end
    endgenerate

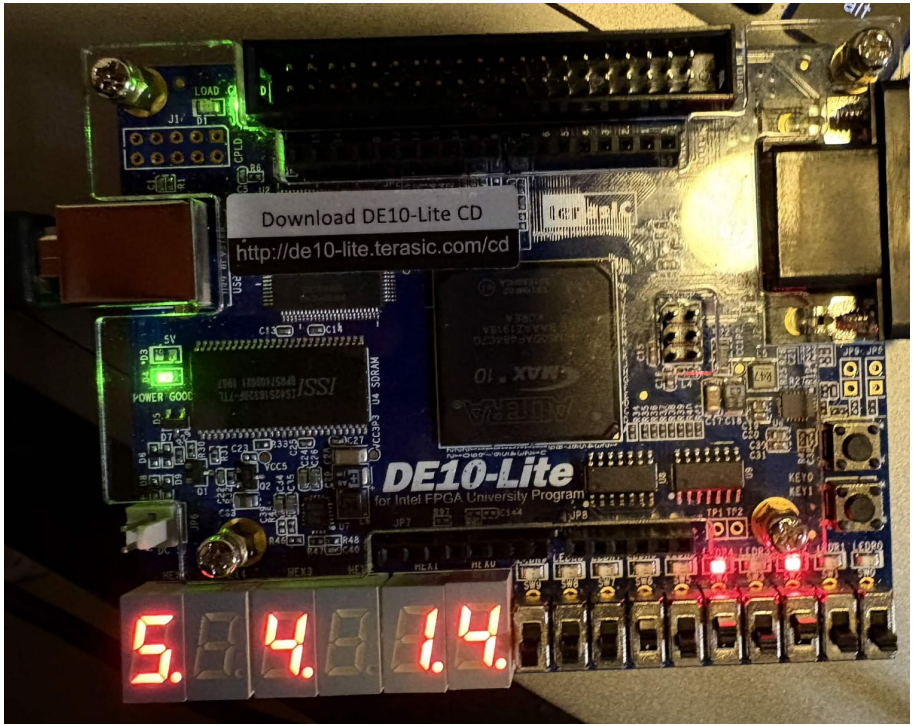
    assign prod[2*n-1:n] = {c_o[n-1], s_row[n-1][n-1:1]};
endmodule
```

```verilog
//Part III
array_multi #(.n(4)) a_multi
(.a(SW[7:4]), .b(SW[3:0]), .prod(LEDR[7:0]));

seg7_display seg5 (.b_num(SW[7:4]), .seg_disp(HEX5));
seg7_display seg3 (.b_num(SW[3:0]), .seg_disp(HEX3));
seg7_display seg1 (.b_num(LEDR[7:4]), .seg_disp(HEX1));
seg7_display seg0 (.b_num(LEDR[3:0]), .seg_disp(HEX0));

assign HEX4 = 8'hFF;
assign HEX2 = 8'hFF;
```

DE10-lite



Functions as intended updates immediately

Part 4

Schematic

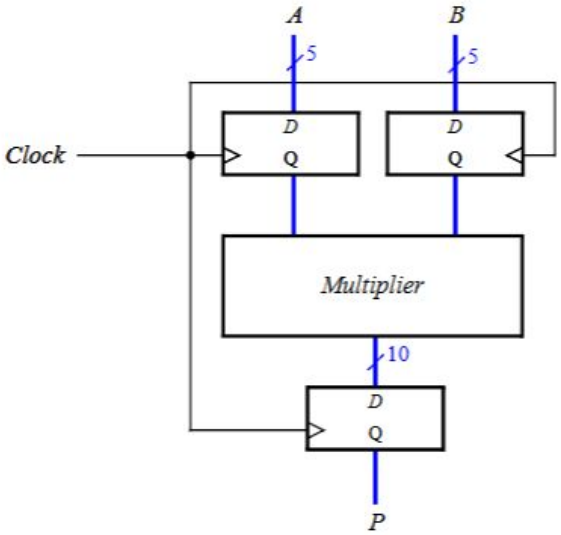
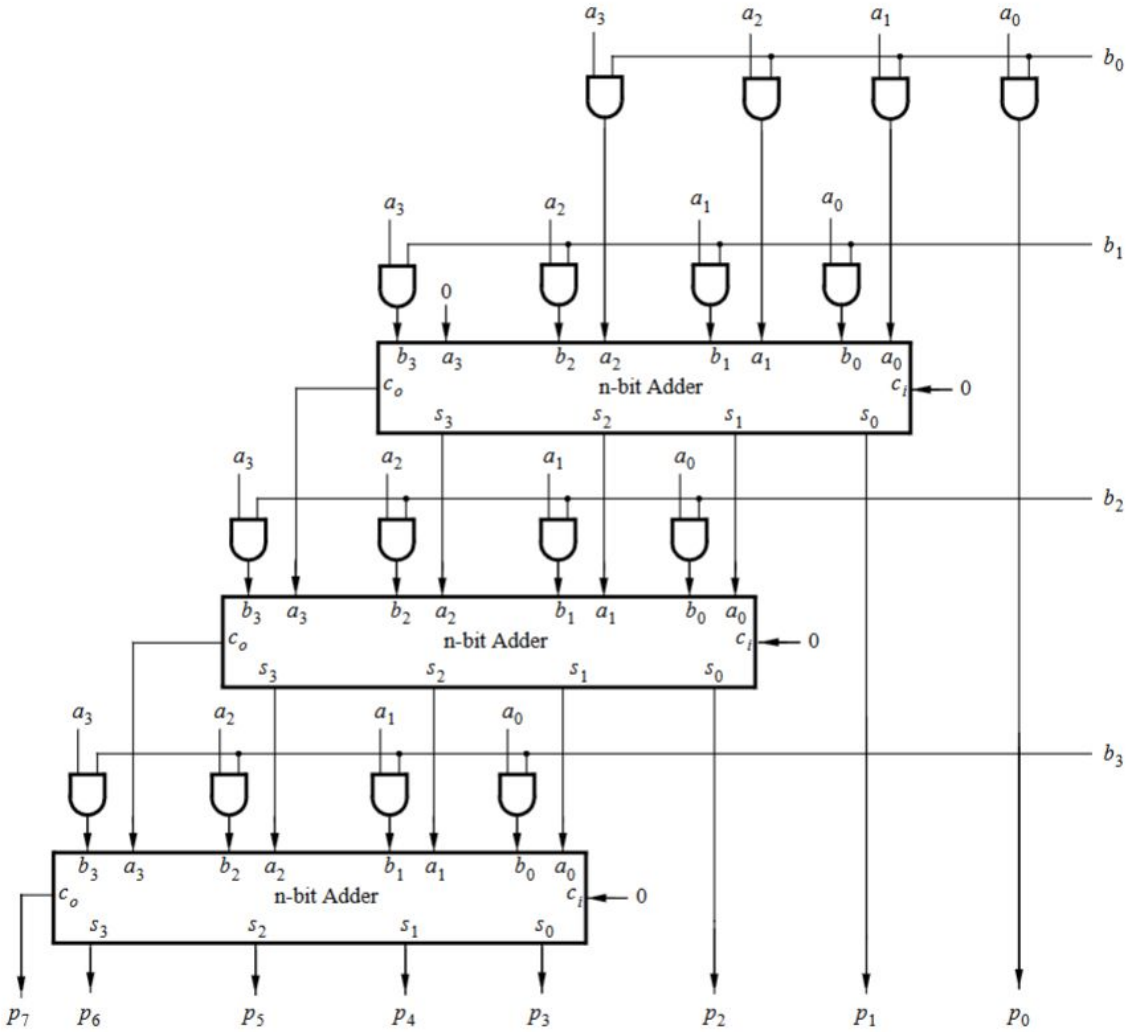


Figure 6: A registered multiplier circuit.



Verilog Code

```
//Part IV
wire [9:0] P;
reg_multi #(.n(5)) r_multi
(.clk(KEY[1]), .a(SW[9:5]), .b(SW[4:0]), .P(P[9:0]));

assign LEDR = SW;

seg7_display seg2 (.b_num({2'b00, P[9:8]}), .seg_disp(HEX2));
seg7_display seg1 (.b_num(P[7:4]), .seg_disp(HEX1));
seg7_display seg0 (.b_num(P[3:0]), .seg_disp(HEX0));

assign HEX5 = 8'hFF;
assign HEX4 = 8'hFF;
assign HEX3 = 8'hFF;
```

With how the circuit is designed 2 KEY1 presses are needed which is similar issue as in part 1

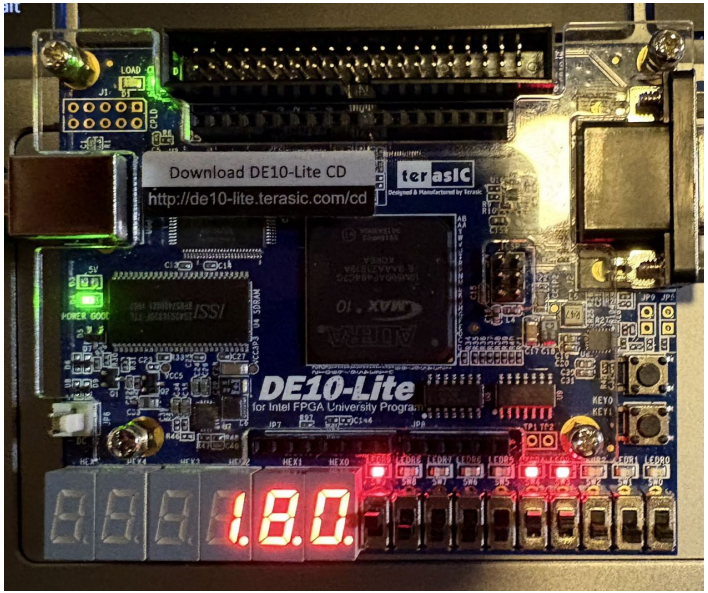
```
module reg_multi #(
    parameter n = 8
)(
    input clk,
    input [n-1:0] a, b,
    output reg [2*n-1:0] P
);

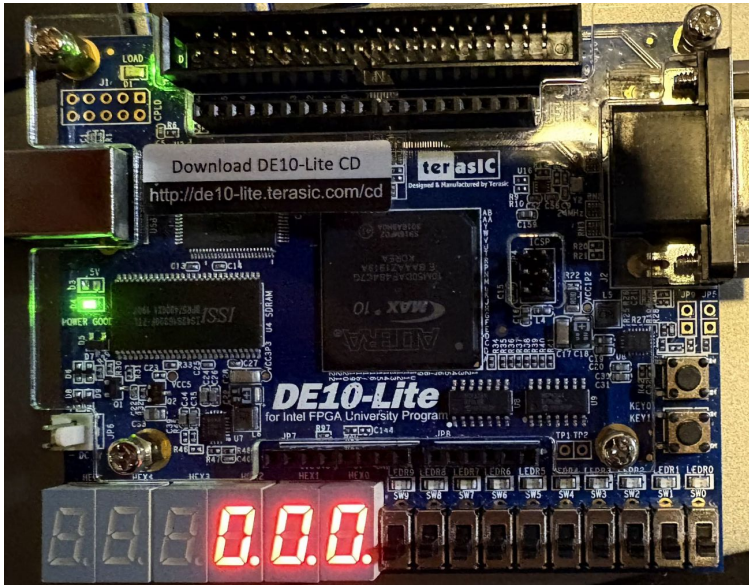
    reg [n-1:0] a_r, b_r;
    wire [2*n-1:0] prod;

    array_multi #(.n(n)) multi
    (.a(a_r), .b(b_r), .prod(prod));

    always @(posedge clk) begin
        a_r <= a;
        b_r <= b;
        P <= prod;
    end
endmodule
```

DE10-lite





After KEY1 pressed twice

Questions

A total of 74 logic elements were used to implement the verilog code

The circuit had a max frequency of 183.05 MHz

Compilation Hierarchy Node		Logic Cells	Dedicated Logic Registers
|main	1	74 (1)	20 (0)
|reg_multi:r_multi|	1	56 (18)	20 (20)
|array_multi:multi|	1	53 (17)	0 (0)
|ripple_a...p[1].row|	1	9 (9)	0 (0)
|ripple_a...p[2].row|	2	9 (9)	0 (0)
|ripple_a...p[3].row|	3	9 (9)	0 (0)
|ripple_a...p[4].row|	4	9 (9)	0 (0)
|seg7_display:seg0|	2	7 (7)	0 (0)
|seg7_display:seg1|	3	7 (7)	0 (0)
|seg7_display:seg2|	4	3 (3)	0 (0)

Slow 1200mV 85C Model Fmax Summary

<<Filter>>

	Fmax	Restricted Fmax	Clock Name
1	183.05 MHz	183.05 MHz	KEY[1]